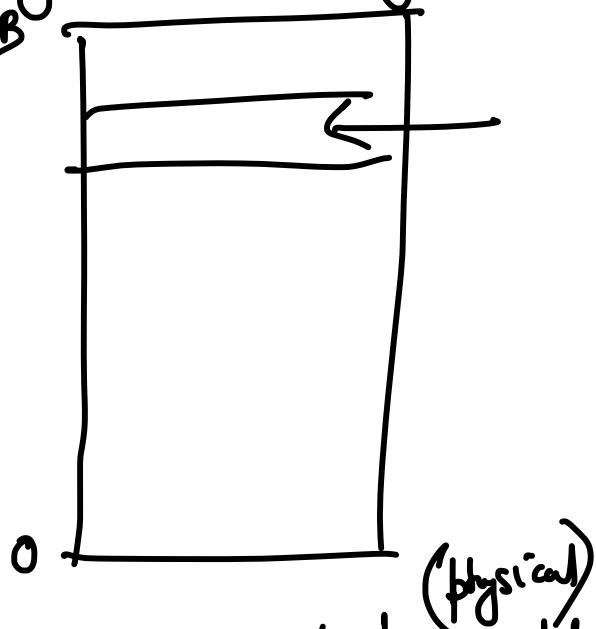


OPERATING SYSTEMS

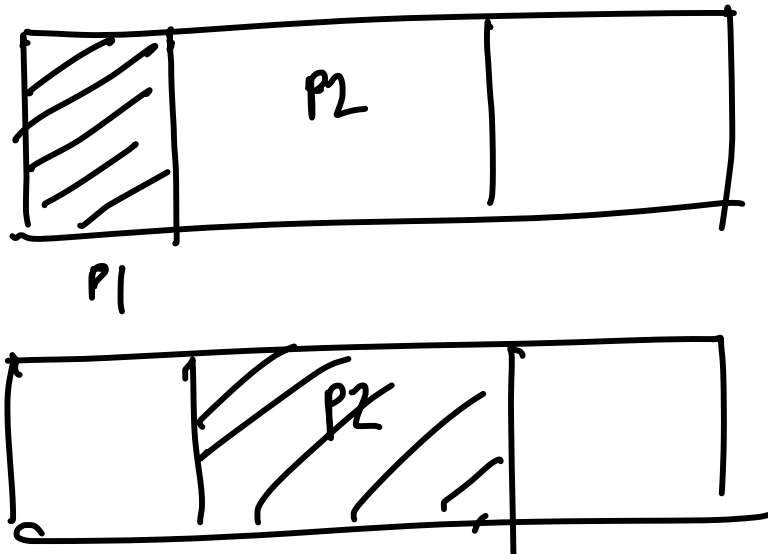
- 1) Assembly Programming
- 2) Process Management [OS is a set of abstractions provided to the user programs.]
→ Process
- 3) Virtual Memory [Memory Management] → File
→ Address Space

Processes are not aware of which part of the memory they are using.



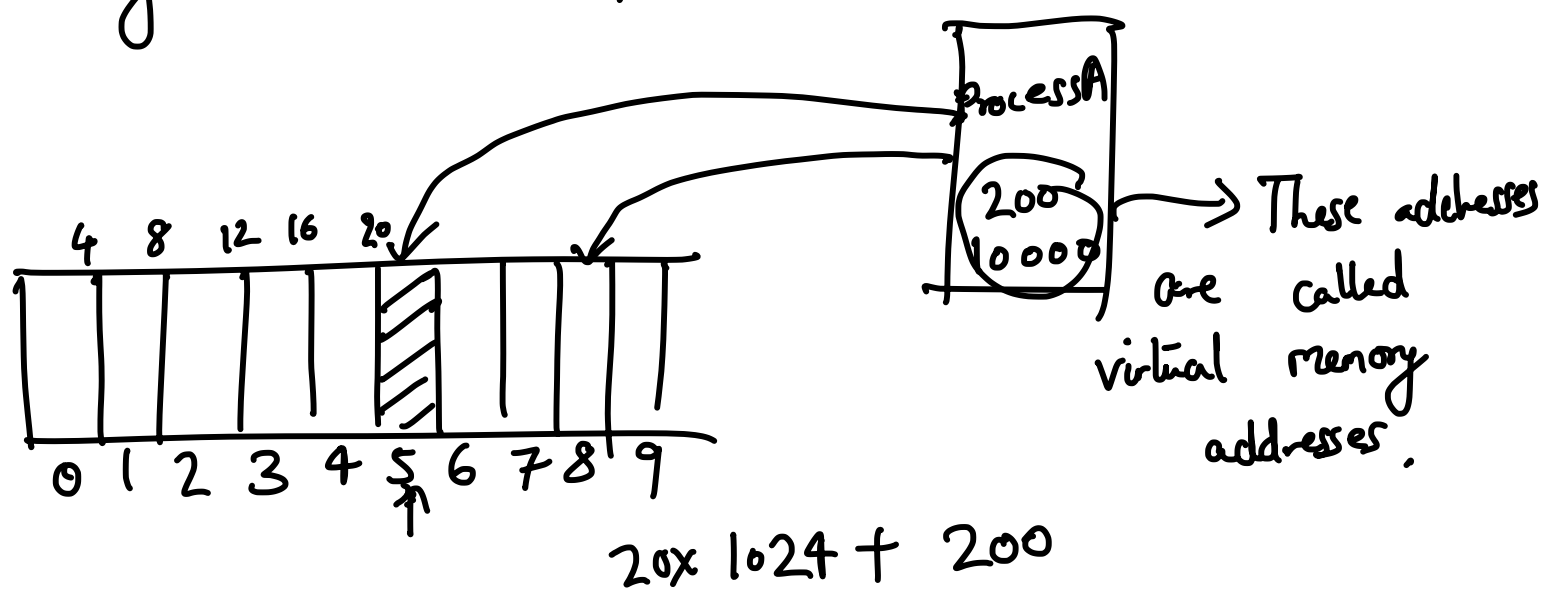
At runtime, the actual (physical) address used needs to be translated. Whenever a process is loaded into the physical memory, we add the address corresponding to 0 at the base register. The physical address is equal to the sum of the base register and the address used by the process.

Limit register which specifies the size of address space.



P3 cannot fit into any of the currently available memory spaces, even though sufficient memory is available. One possible way is compaction, where you move all the processes to fit them in memory. This hurts the starting time of processes.

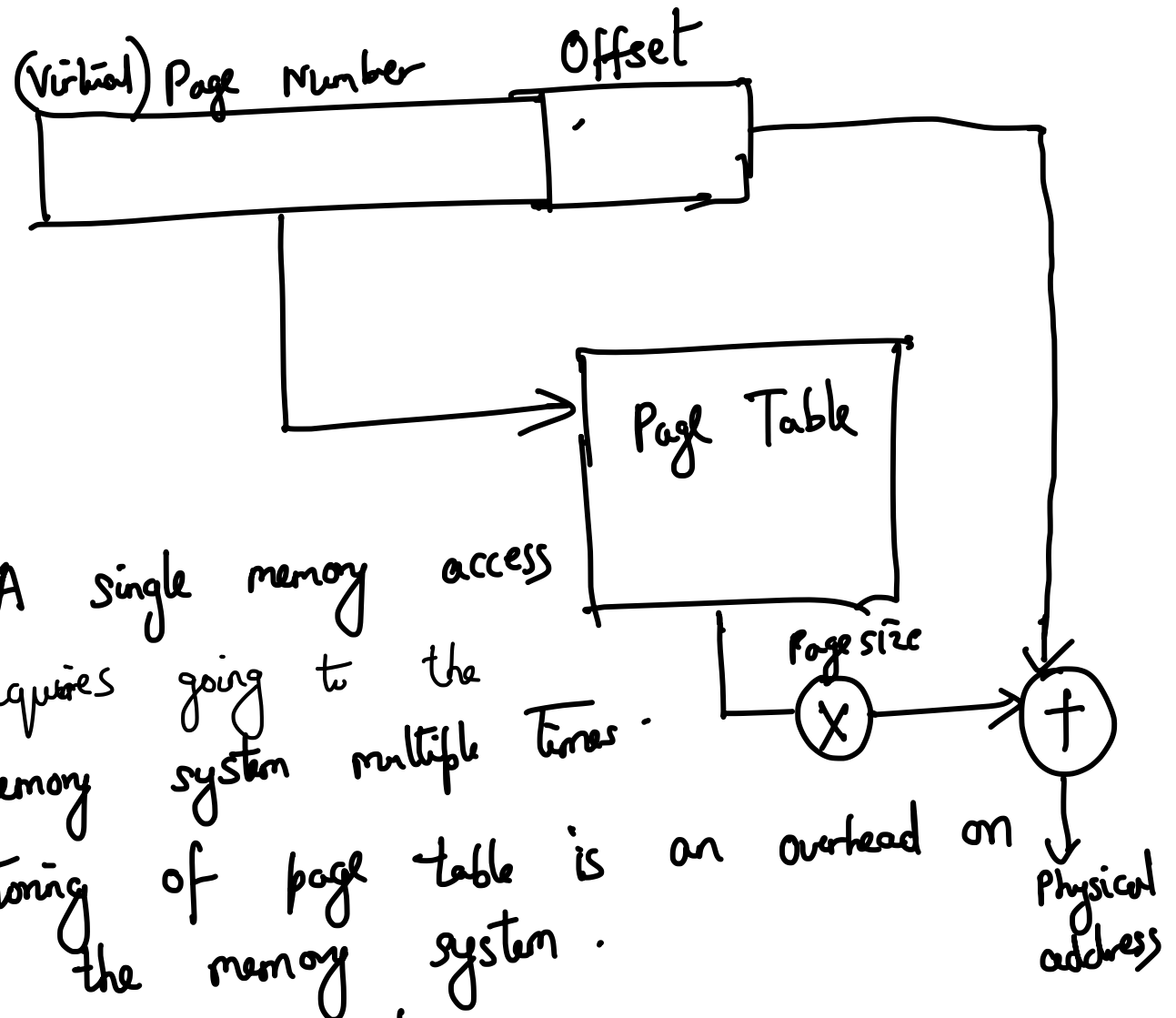
Paging → You divide the entire memory into pages of smaller units (like 4KB). Allocate memory to processes only in units of pages.



You need a page table which keeps this information

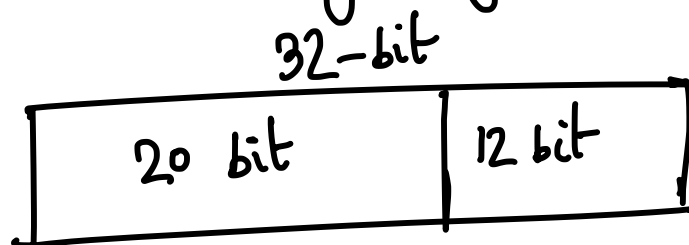
Page number	Frame number	FLAGS

The address used by the process is called virtual address. Virtual address is divided into two parts.



1) A single memory access requires going to the memory system multiple times.

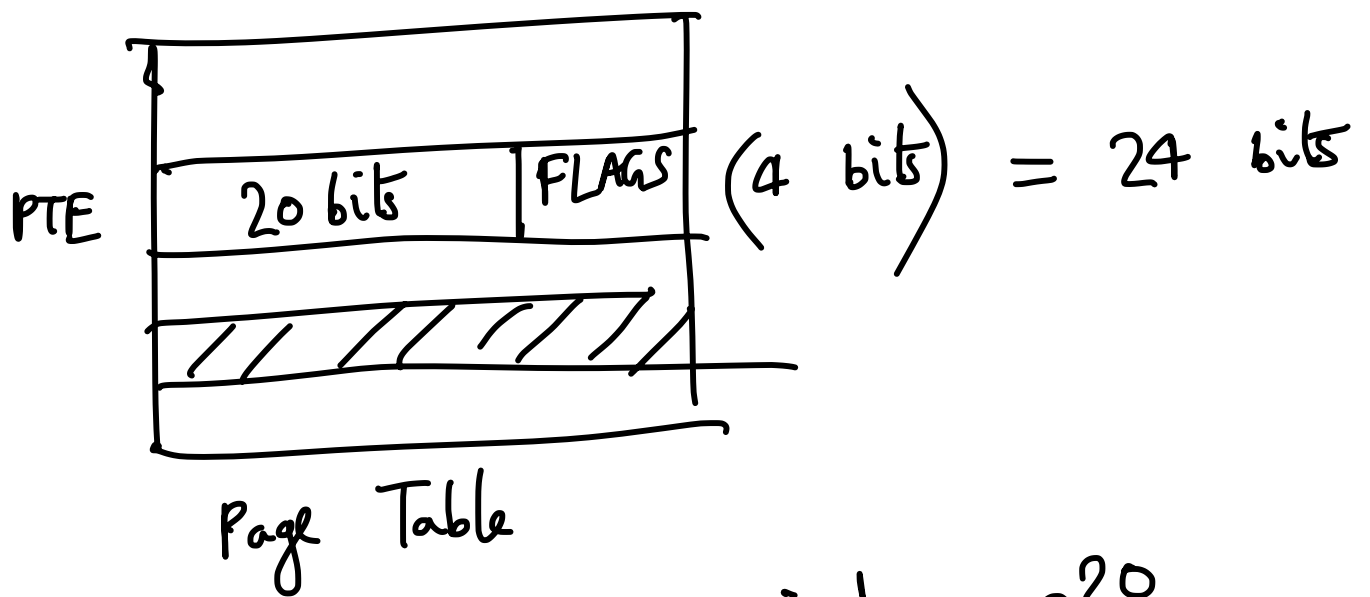
2) Storing of page table is an overhead on the memory system.



In the page table, you need to store the physical address. Assume you have 4 GB of RAM. How many page frames can be present in the main memory?

$$\frac{4 \text{ GB}}{4 \text{ KB}} = \frac{2^{32}}{2^{12}} = 2^{20}$$

(Can be addressed in 20 bits)



Total number of entries required = 2^{20}

Size of page Table = $2^{20} * 24 \text{ bits}$

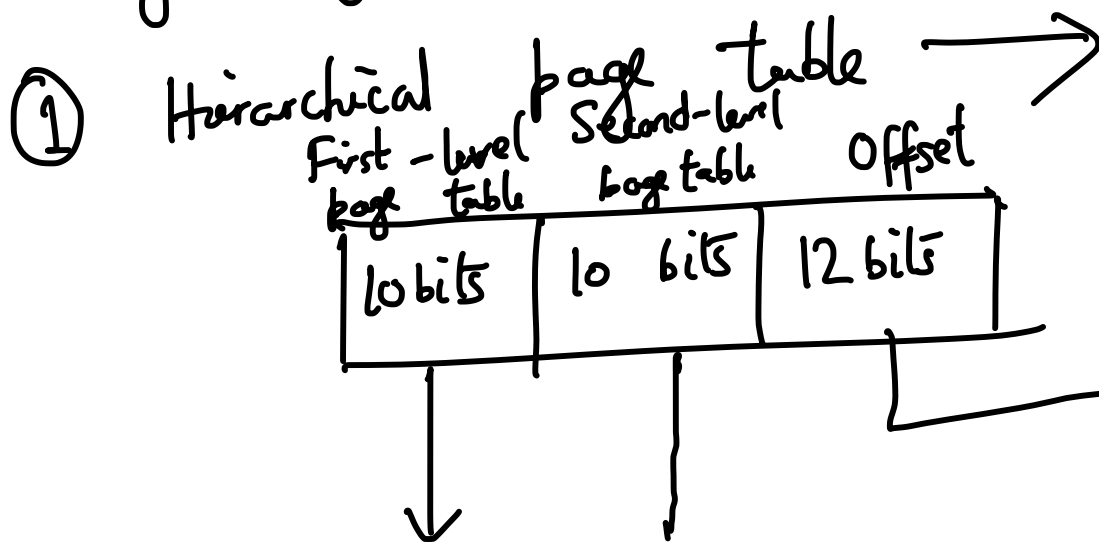
= $3 * 2^{20} \text{ bytes}$

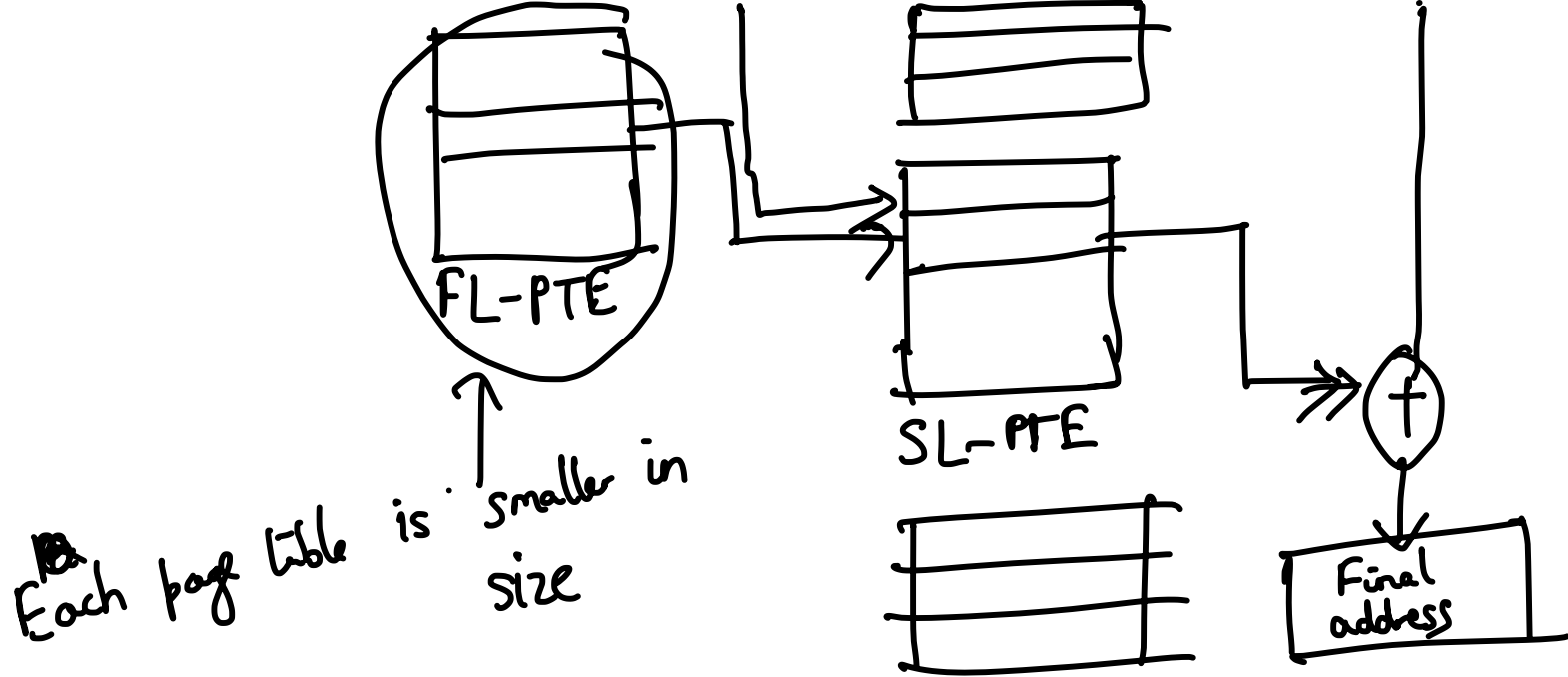
= 3 MB

(which is significant amount of memory)

$2^{36} * 36 \text{ bits}$
 $\approx 2 \text{ GB's of memory}$
 (cannot be stored in main memory).

The amount of overhead can be very large in today's systems.





Each page table is smaller in size

Hierarchical page tables allow us to have very long virtual addresses. The tradeoff is that it requires more number of memory accesses.

What is the solution?

→ A new cache memory is kept called Translation Lookaside Buffer (TLB). If a hit happens, then the physical address can be obtained directly from the TLB. Compared to traditional cache memory, much higher hit rates are possible. This can easily avoid the very high number of memory accesses in hierarchical page tables.

First-level		Second-level	Physical Page Frame Number
page no.	page no.		

2) Inverted Page Table → HashMaps ...

How to handle a case of no physical page frame is allocated? (usually handled in software)

by indicating the memory management to
rearrange the memory).